



Proudly supported by



edunet
foundation



Module - 4

Building Apps with the ABAP RESTful Application Programming Model



DISCLAIMER

The content is curated from online/offline resources and used for educational purpose only.

Learning Objectives

- RAP Architecture Overview
- Involved Technologies
- SAP Cloud Platform ABAP Environment
- Practical Examples
- Advanced Techniques
- Summary



Source

Learning Outcome

- Understand the purpose and architecture of RAP
- Distinguish between managed and unmanaged BOs
- Create CDS views and define BO behavior
- Expose services via OData using service definitions
- Build and test simple RAP apps using ADT
- Use EML and annotations for Fiori integration
- Develop cloud-ready, stateless ABAP applications



Source



What is RAP?

ABAP RESTful Application Programming Model (RAP)

- **Strategic long-term ABAP development model**

RAP is SAP's modern and sustainable approach to ABAP development, designed to provide a consistent, future-ready framework for building enterprise applications with high code quality and maintainability.

- **Suitable for cloud & on-premise**

RAP supports both SAP S/4HANA (on-premise) and the SAP Business Technology Platform (cloud), allowing developers to create applications that are adaptable to various deployment environments without changing the development approach.

- **Unified model for SAP Fiori & Web APIs**

With RAP, developers can build both SAP Fiori user interfaces and OData-based Web APIs using a single, integrated set of tools and concepts—streamlining development and ensuring consistency across front-end and service layers.

Key Principles of RAP

Principle	Description
User Experience	SAP Fiori, browser/device responsive
Cloud Readiness	Stateless, scalable
Code Efficiency	ABAP SQL, CDS, ADT
Quality & Security	Testable, supportable apps

PROVIDE A
PROGRAMMING
MODEL ...

... AS A STRATEGICAL LONG-TERM SOLUTION FOR ABAP DEVELOPMENT

... FOR THE EFFICIENT DEVELOPMENT OF

SAP Fiori apps and Web APIs,
from scratch or **by integrating legacy code**

... OFFERING AN END-TO-END DEVELOPMENT EXPERIENCE WITH

standardized development flow

best practices & development guides

high development **efficiency**

focus on business logic, rather than technical aspects

native **testability**, **documentability**, and **supportability**

code pushdown to SAP HANA

comfortable support for **stateless** and **stateful** environments

... SUPPORTING THE PRODUCT QUALITIES

User Experience: SAP Fiori and SAP HANA

Cloud: scalability

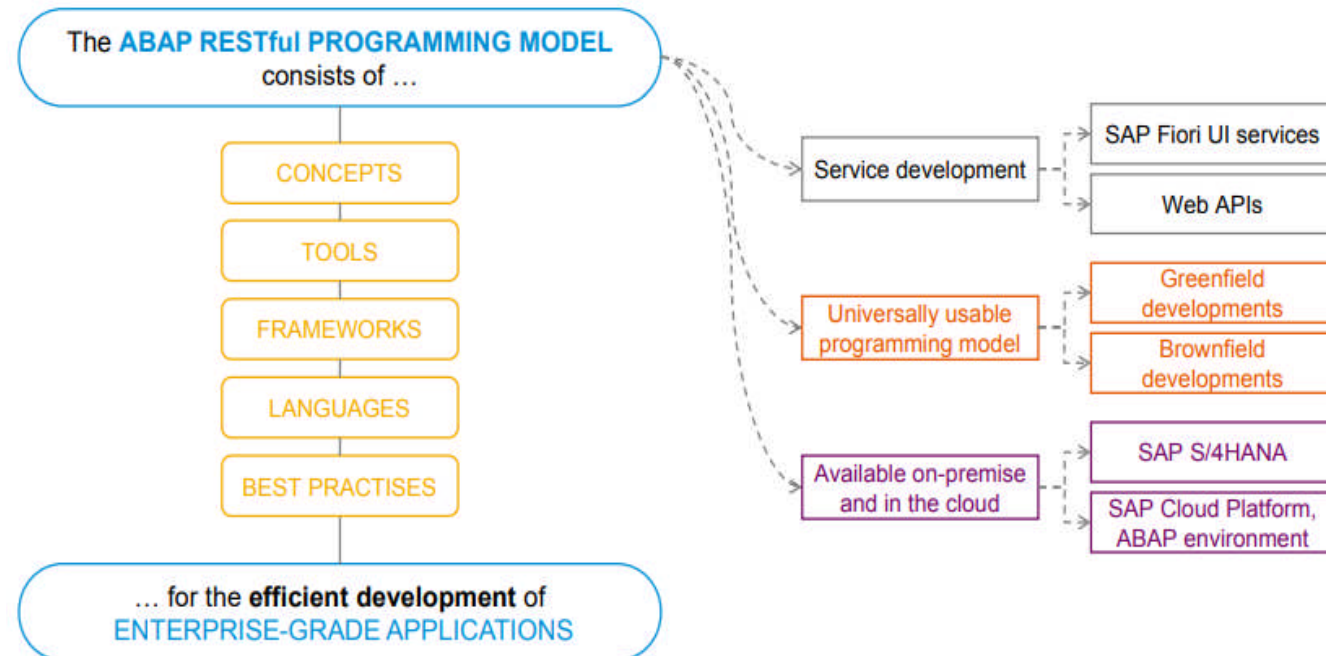
Flexibility: break-outs for non-standardized implementations

Out-of-the-box extensibility and verticalization

What is RAP?

ABAP RESTful Application Programming Model (RAP)

- Strategic long-term ABAP development model
- Suitable for cloud & on-premise
- Unified model for SAP Fiori & Web APIs



End-to-End Development Flow

- **Based on ABAP Development Tools (ADT)**

The entire RAP development process is carried out using ADT in Eclipse, offering a modern and integrated environment for efficient ABAP programming.

- **Static code checks, auto-completion**

Developers benefit from built-in features like real-time syntax validation, code suggestions, quick fixes, and automated checks to ensure clean and secure code.

- **CDS & ABAP integration**

Core Data Services (CDS) views are seamlessly integrated with ABAP code, allowing declarative data modeling and logic implementation within a unified toolset.



ABAP Development Tools in Eclipse for all development tasks

Easy developer onboarding
End-to-end development flow



Languages: ABAP and CDS

Standard implementation tasks via typed APIs supporting
static code checks, auto-completion, element info



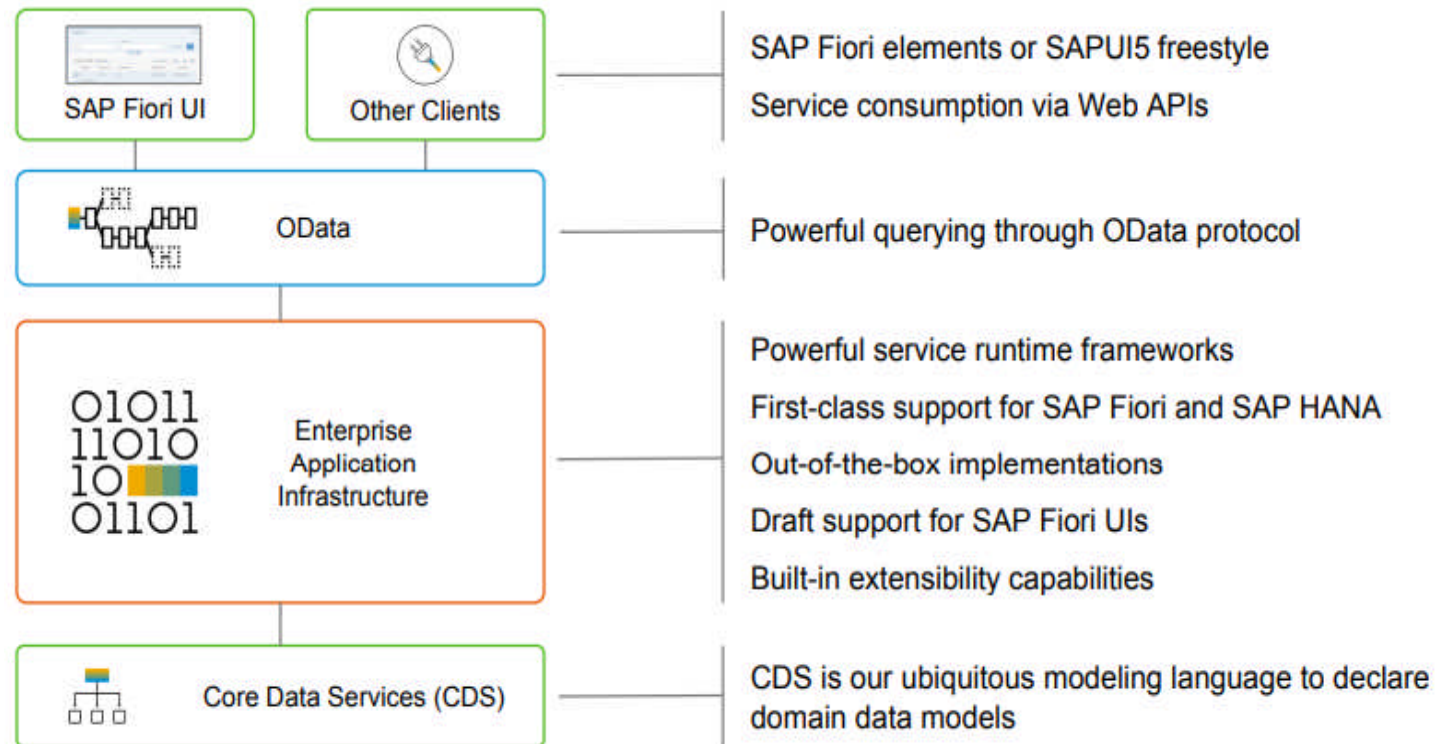
Powerful frameworks

Take over technical implementation tasks
Business logic added in code exits on protocol-agnostic layers

RAP Core Components

Key Players

- CDS Views
- Behavior Definitions (BDEF)
- Service Definitions
- Service Bindings
- SAP Fiori Elements



RAP Architecture Layers

- **Data Modeling (CDS Views)**
Defines the structure and relationships of data using Core Data Services, providing a semantic, reusable data model.
- **Behavior (BDEF + ABAP logic)**
Specifies how data can be manipulated (create, update, delete) and includes business logic like validations and actions.
- **Service Exposure (Service Definitions + Bindings)**
Makes the business logic and data available to UIs or APIs via OData, defining how the app is consumed.

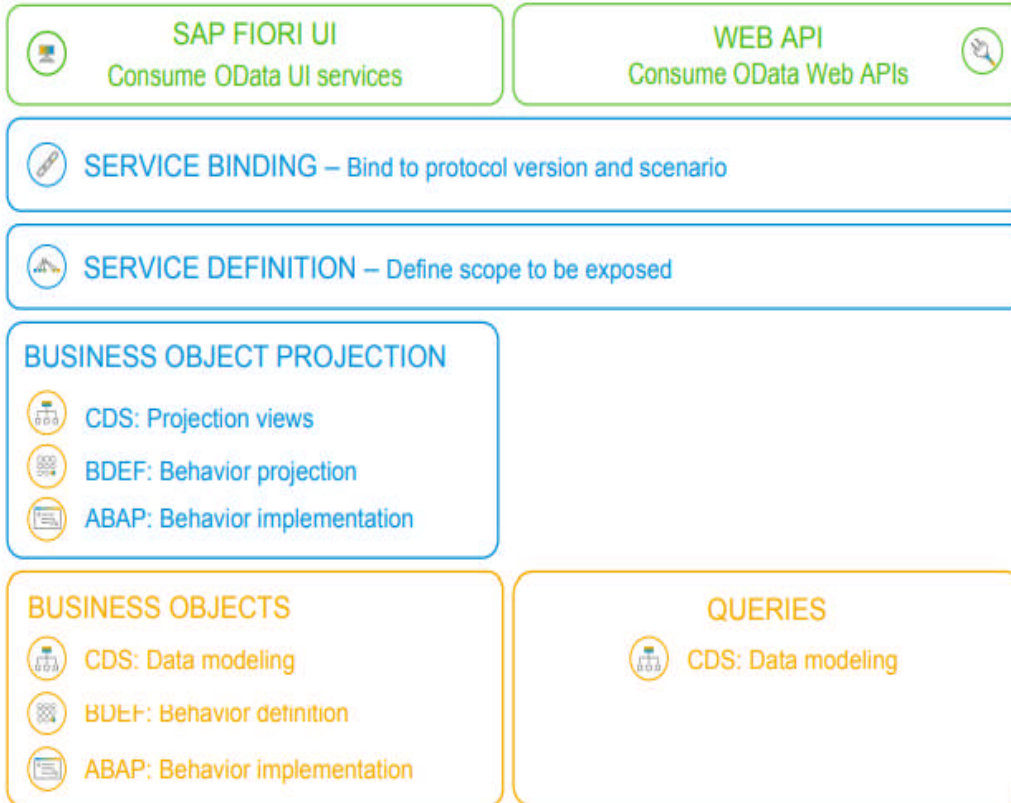
SERVICE
CONSUMPTION



BUSINESS
SERVICES
PROVISIONING



DATA MODELING
& BEHAVIOR



What is a Business Object?

- **CDS root and composition tree**

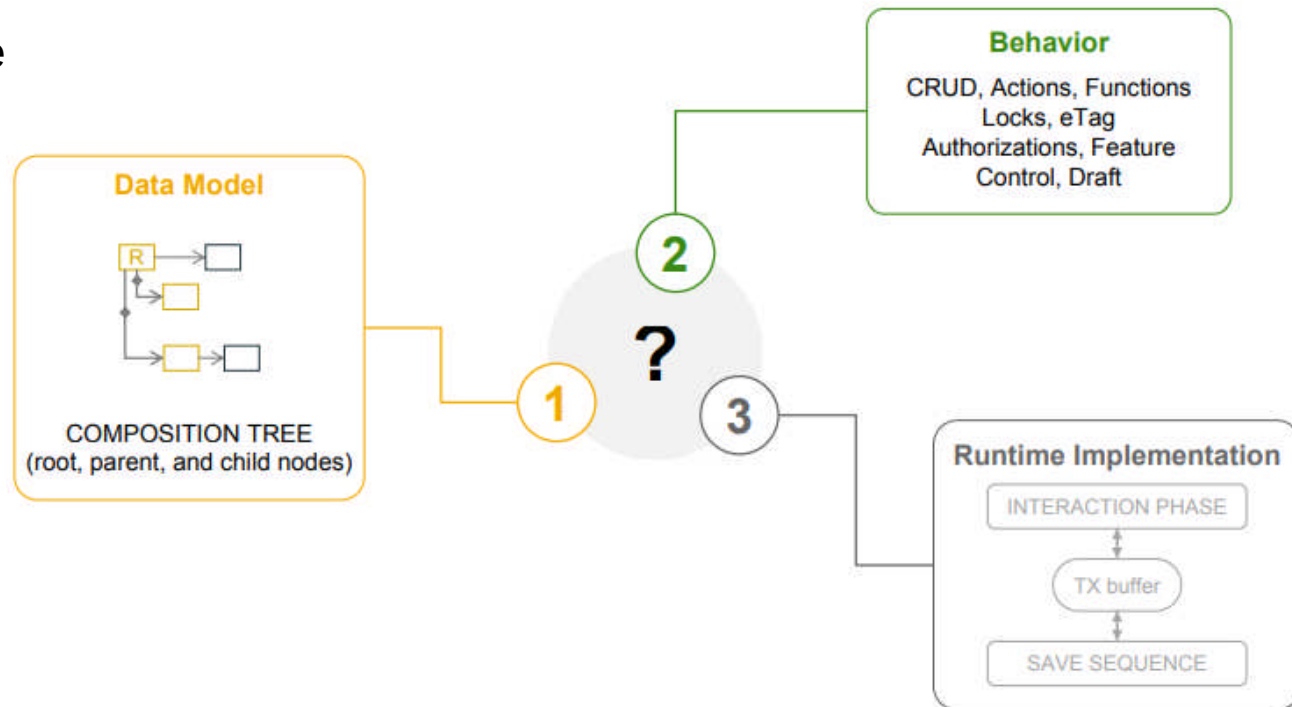
Defines a hierarchical data structure where one root entity (like a sales order) is linked to one or more related child entities (like items).

- **Behavior definition for operations (CRUD, actions)**

Specifies how the business object can be manipulated — Create, Read, Update, Delete — along with custom actions and validations.

- **Draft functionality for stateless apps**

Allows temporary data storage during user input, enabling features like Save as Draft in stateless, cloud-ready applications.





Managed vs Unmanaged BOs

- **Managed BO**

The RAP framework handles standard operations like create, update, delete, and draft logic automatically. Ideal for new (greenfield) applications with minimal custom code.

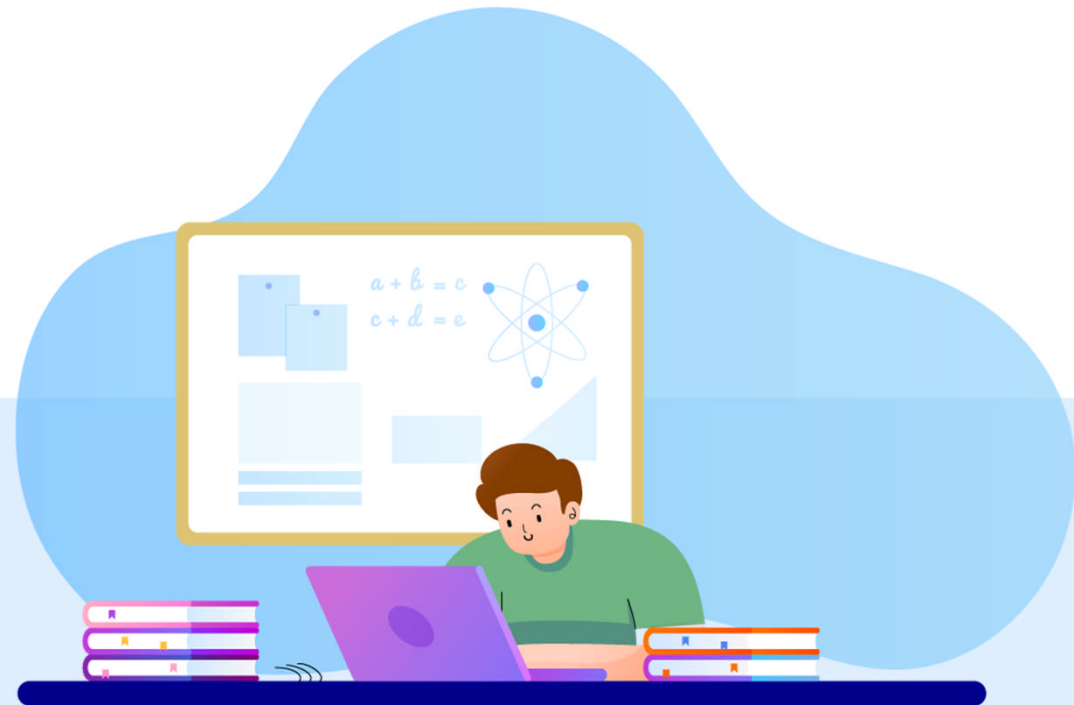
- **Unmanaged BO**

The developer implements full control over all operations (CRUD, actions, validations). Suitable for reusing existing business logic from legacy applications (brownfield approach).

Type	Use Case	Who Handles CRUD?
Managed	Greenfield	RAP Framework
Unmanaged	Legacy code reuse	Developer

Lab

- Practical Implementation of RAP BO



Source

<https://www.freepik.com/>



Technologies Behind RAP

- **SAP HANA (in-memory, fast aggregations)**
A high-performance database that processes data in-memory, enabling real-time analytics and fast transaction processing.
- **ABAP Core Data Services (CDS)**
A powerful data modeling layer in ABAP used to define semantic views directly on database tables with annotations and logic.
- **Modern ABAP Language Features**
Includes inline declarations, expression-based coding, table operations, and better support for unit testing and code clarity.
- **Modern ABAP Language Features**
Includes inline declarations, expression-based coding, table operations, and better support for unit testing and code clarity.

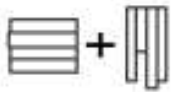
ABAP development for SAP HANA

KEY TECHNOLOGY ASPECTS OF SAP HANA



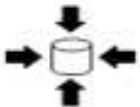
IN-MEMORY COMPUTING

Allowing OLTP and OLAP in real time



ROW AND COLUMN-BASED DATA STORE

Very fast aggregation and search



HIGH DATA COMPRESSION

Make use of real-life / sparse fill of tables



MULTICORE ARCHITECTURE SUPPORT

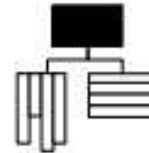
Benefit from massive parallelization

KEY IMPACTS ON ABAP APP DEVELOPMENT



CODE PUSHDOWN

Delegation of data computing to the DB



NO AGGREGATES

On-the-fly data models without duplicates



FEWER INDICES

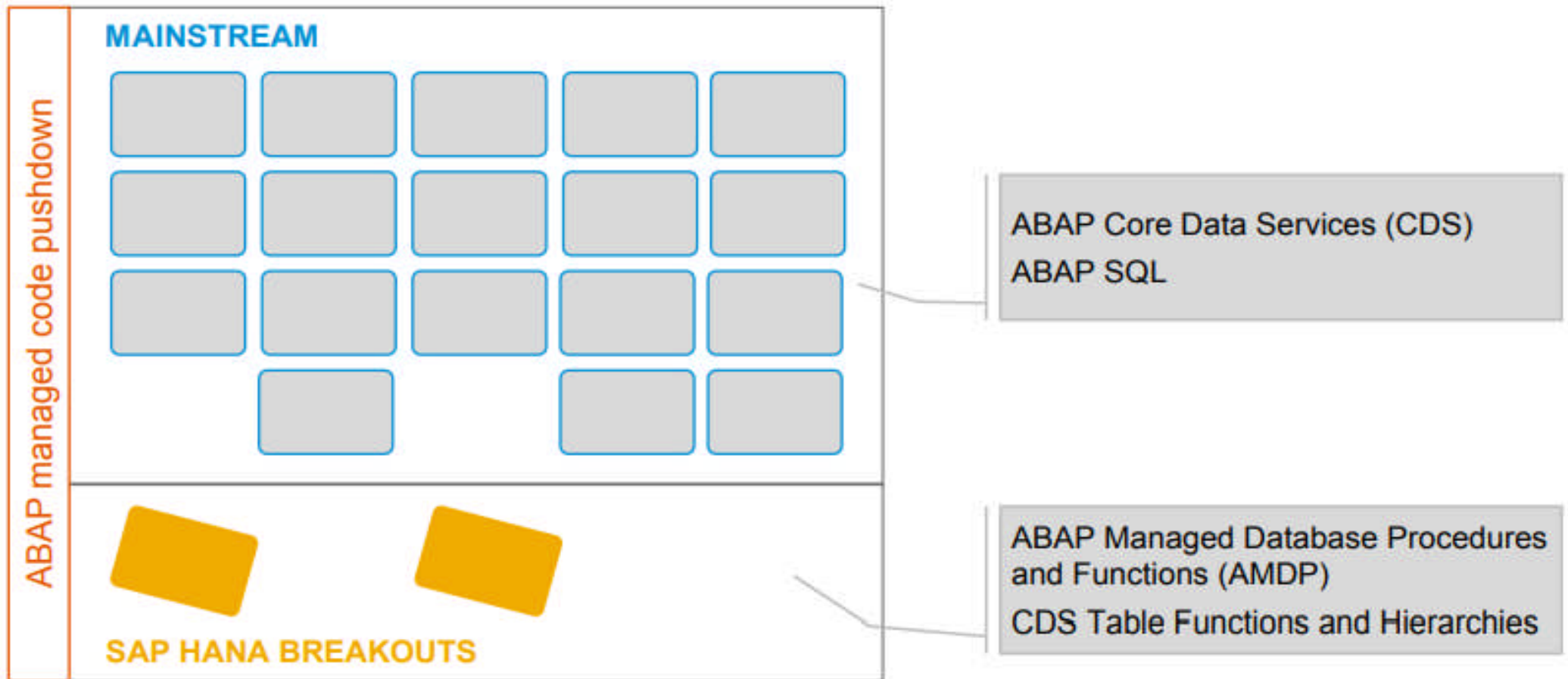
Flexible and fast retrieval of the dataset



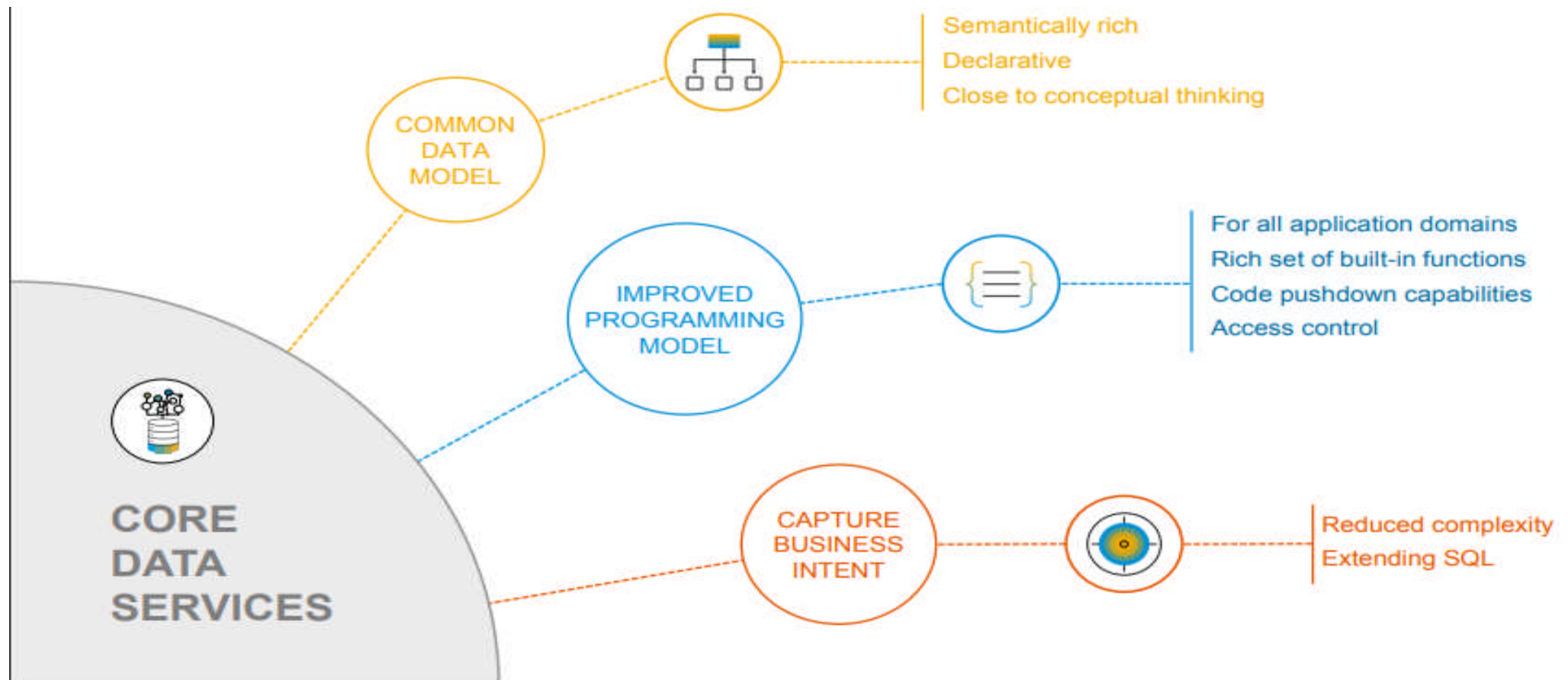
FEWER CODE LINES

Less complexity in data models and code

ABAP development for SAP HANA: Mainstream and Code Breakouts



ABAP Core Data Services (CDS) – Next-generation Data Modeling





CDS View Syntax

```
@EndUserText.label: 'Customer View'           // Optional: Label for the view
define view entity ZI_Customer                  // Define a new CDS view entity
  as select from snwd_bpa                       // Source table: snwd_bpa (business partners)
{
  key businesspartner as BusinessPartnerID,    // Primary key field renamed for clarity
    companyname      as CompanyName,          // Select and rename company name
    emailaddress     as Email                  // Select and rename email address
}
```



Entity Manipulation Language (EML)

" Declare an internal table to hold instances of the Travel BO (business object)
DATA lt_travel TYPE TABLE OF zbp_travel.

" Fill the internal table with a new travel record using the VALUE constructor

```
lt_travel = VALUE #(
  ( TravelID = 'T123'           " Unique ID for the travel
    AgencyID = 'AG001'         " Travel agency identifier
    CustomerID = 'C001'        " Customer identifier
  )
).
```

" Use Entity Manipulation Language (EML) to create the business object instance

CALL METHOD cl_abap_behavior_handler=>create

EXPORTING

it_data = lt_travel. " Pass the internal table as input data for creation



Advanced RAP Example: Draft Enabled BO

- **CDS View Definition (Draft Enabled)**

@AccessControl.authorizationCheck: #NOT_REQUIRED	" No authorization checks applied for simplicity (not recommended for production)
@EndUserText.label: 'Draft Travel Entity'	" User-friendly label for the CDS view
define root view entity ZI_TravelDraft	" Define the root CDS view entity for the draft-enabled BO
as select from ZTRAVEL_DRAFT	" Source is the draft-enabled table
{	
key TravelID,	" Unique identifier for each travel record
AgencyID,	" Travel agency responsible
CustomerID,	" Customer associated with the travel
BeginDate,	" Start date of the travel
EndDate	" End date of the travel
}	



Advanced RAP Example: Draft Enabled BO

- **Behavior Definition (with Draft Support)**

define behavior for ZI_TravelDraft	" Define behavior for the CDS view ZI_TravelDraft
persistent table ZTRAVEL_DRAFT	" Link to the database table where data is stored
with draft	" Enable draft support for this business object
{	
create;	" Allow creation of records
update;	" Allow updating records
delete;	" Allow deleting records
draft action Activate;	" Finalize draft changes and activate the data
}	

Modern ABAP Features

- **Inline declarations: DATA(x) = ...**
Allows you to declare and assign variables in a single line, making code more concise and readable.
- **Table expressions: VALUE, CORRESPONDING**
VALUE is used to construct structured data or internal tables easily, while CORRESPONDING maps fields between structures with matching names.
- **ABAP Doc & Unit Testing**
ABAP Doc enables inline documentation of methods and classes, while ABAP Unit allows for writing automated tests to ensure code correctness and stability.

MODERN ABAP

Simple and concise ABAP code through new language features like inline declarations, constructor expressions

Extensively expression-oriented syntax

Advanced table operations like CORRESPONDING() operator, grouping and filtering

Entity Manipulation Language (EML) to control the transactional business object behavior in the RAP context

JSON support in sXML library

Inline code documentation with ABAP Doc

ABAP Unit Testing with test doubles and test seams

Modern ABAP & EML in the ABAP RESTful Application Programming Model

SERVICE CONSUMPTION



BUSINESS SERVICES PROVISIONING



DATA MODELING & BEHAVIOR



SAP FIORI UI
Consume OData UI services



WEB API
Consume OData Web APIs



SERVICE BINDING – Bind to protocol version and scenario



SERVICE DEFINITION – Define scope to be exposed

BUSINESS OBJECT PROJECTION



CDS: Projection views



BDEF: Behavior projection



ABAP: Behavior implementation

BUSINESS OBJECTS



CDS: Data modeling



BDEF: Behavior definition



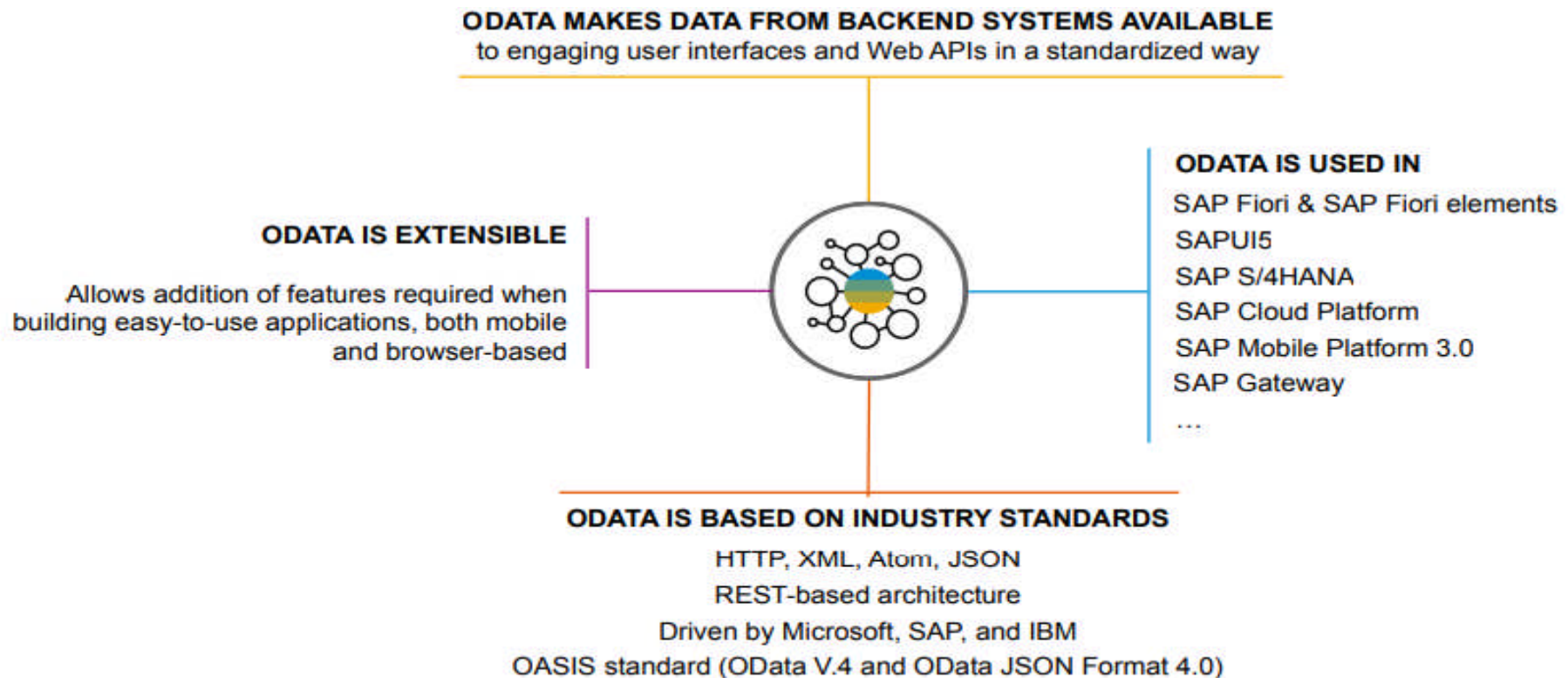
ABAP: Behavior implementation

QUERIES



CDS: Data modeling

The Open Data Protocol (OData) as service protocol for simplified access to SAP data



Creating A Future-ready Workforce

Tools: ABAP Development Tools (ADT)

- Based on Eclipse
- Auto-complete, syntax highlighting
- Code checks (ATC, CVA)
- Integrated testing and debugging

MODERN DEVELOPMENT TOOLSET

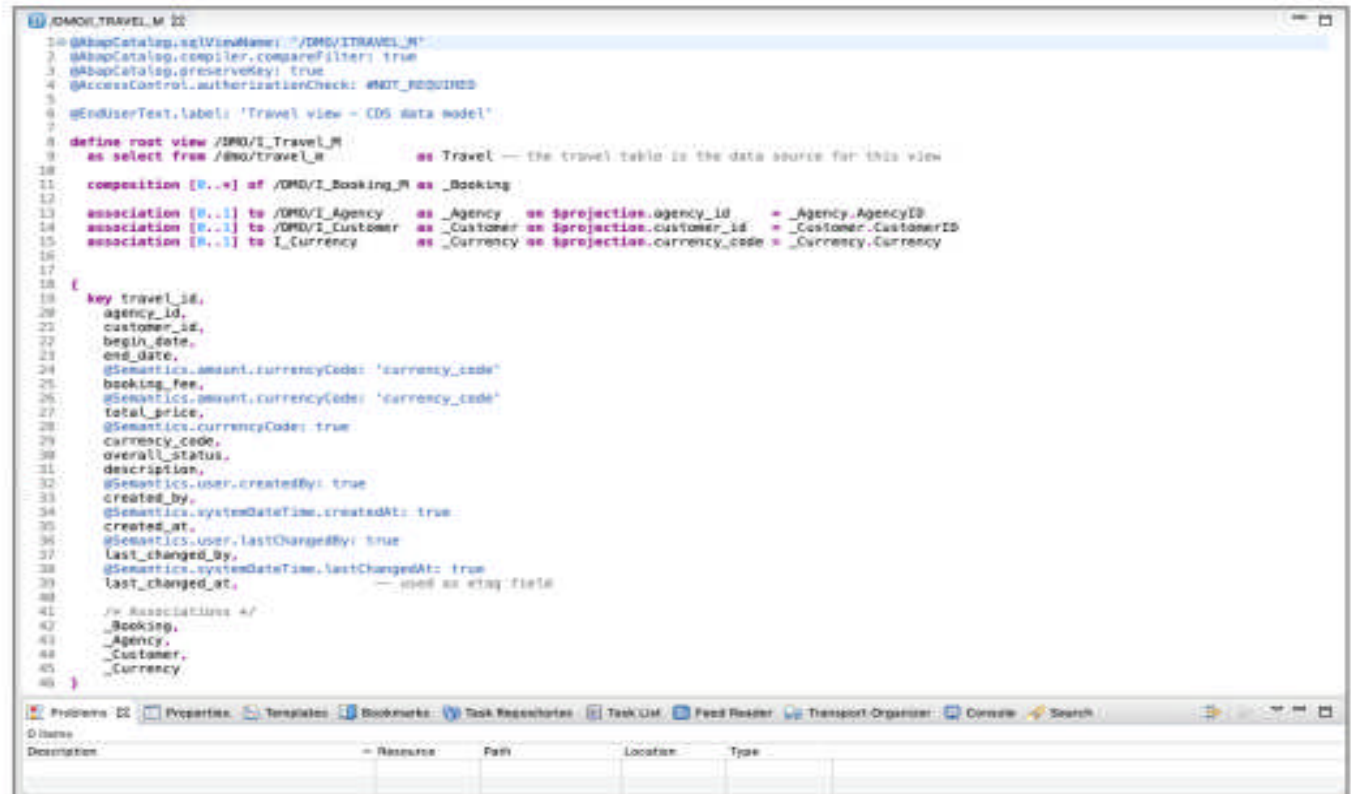
Fully Eclipse-based
Syntax check, Code completion
Syntax highlighting, Pretty printing
Navigation, Search, Quick Fixes

QUALITY ASSURANCE

Static code checks (ATC, CVA) with
remote and local scenarios
Unit testing incl. isolation frameworks
Test seams and injections

SUPPORTABILITY

Debugging, profiling
Static and dynamic logging
Runtime monitoring and analysis



```
1 @ABAPCatalog.sqlViewName: "/IWD/IWTRAVEL_M"
2 @ABAPCatalog.compiler.compareFilter: true
3 @ABAPCatalog.preserveKey: true
4 @AccessControl.authorizationCheck: NOT_REQUIRED
5
6 @EndUserText.label: 'Travel view - CDS data model'
7
8 define root view /IWD/IWTRAVEL_M
9   as select from /BMO/Travel_M      as Travel -- the travel table is the data source for this view
10
11   composition [0..*] of /IWD/IWBooking_M as Booking
12
13   association [0..1] to /IWD/IWAgency as _Agency as $projection.agency_id = _Agency.AgencyID
14   association [0..1] to /IWD/IWCustomer as _Customer as $projection.customer_id = _Customer.CustomerID
15   association [0..1] to I_Currency as _Currency as $projection.currency_code = _Currency.Currency
16
17
18   key travel_id,
19     agency_id,
20     customer_id,
21     begin_date,
22     end_date,
23     @Semantics.amount.currencyCode: 'currency_code'
24     booking_fee,
25     @Semantics.amount.currencyCode: 'currency_code'
26     total_price,
27     @Semantics.currencyCode: true
28     currency_code,
29     overall_status,
30     description,
31     @Semantics.user.createdBy: true
32     created_by,
33     @Semantics.systemDateTime.createdAt: true
34     created_at,
35     @Semantics.user.lastChangedBy: true
36     last_changed_by,
37     @Semantics.systemDateTime.lastChangedAt: true
38     last_changed_at, -- used as etag field
39
40   /> Associations </
41   Booking,
42   _Agency,
43   _Customer,
44   _Currency
45 }
```



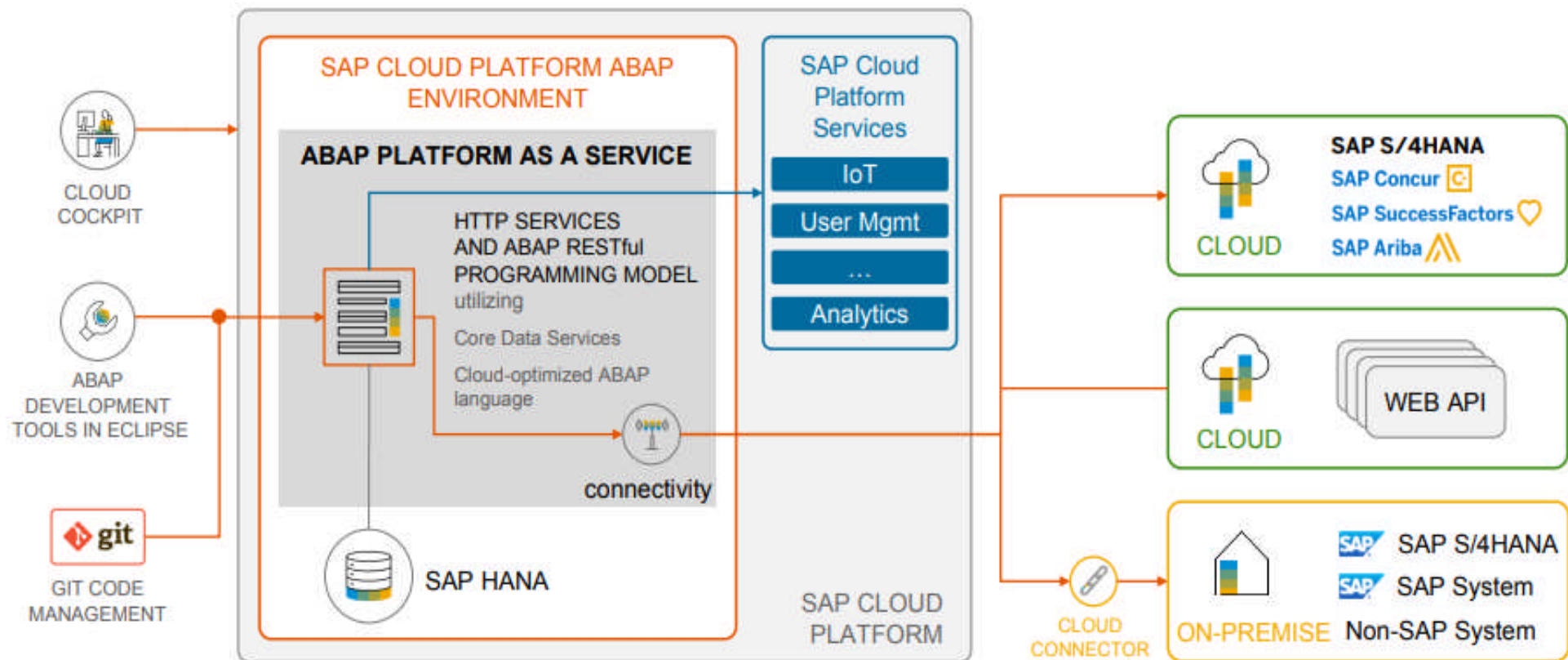
SAP Cloud Platform ABAP Environment

- **The vital parts**
Includes the ABAP runtime, SAP HANA database, development tools (ADT), and connectivity options — all hosted in the SAP BTP cloud infrastructure.
- **Offers ABAP as a PaaS**
Provides ABAP as a Platform-as-a-Service, allowing developers to build and run ABAP applications entirely in the cloud without managing infrastructure.
- **Extends S/4HANA Cloud**
Enables developers to build custom extensions that integrate with SAP S/4HANA Cloud using released APIs, without modifying the core system.
- **Supports side-by-side extensions**
Allows decoupled ABAP applications to run in parallel to core systems, supporting clean core principles and easier upgrades.

Creating A Future-ready Workforce

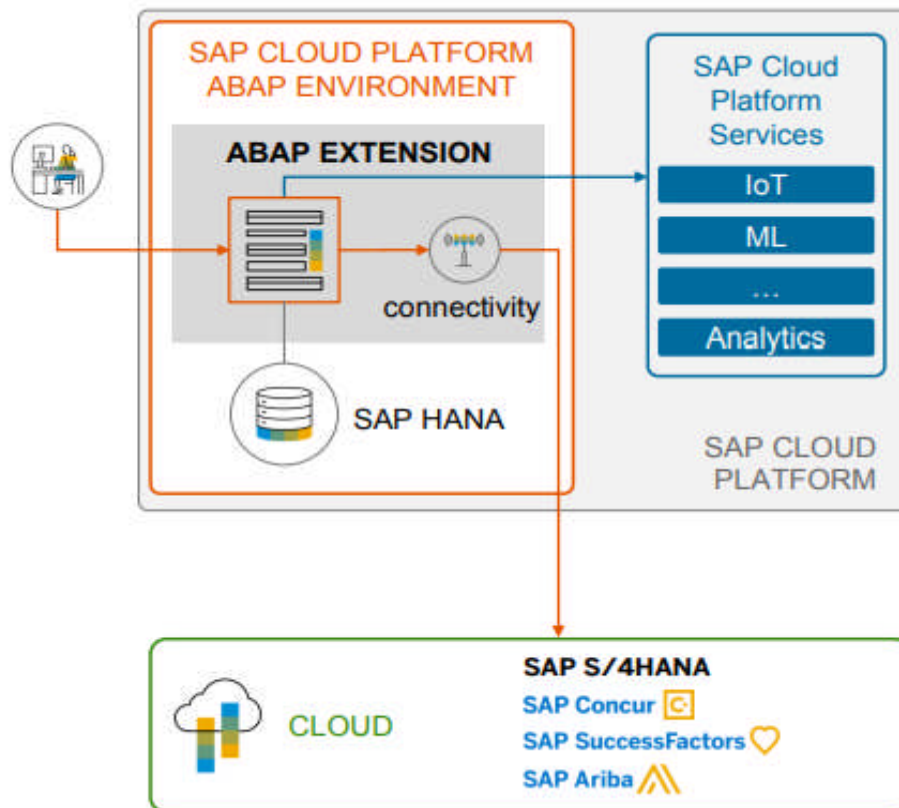
SAP Cloud Platform ABAP Environment

- The vital parts



SAP Cloud Platform ABAP Environment

- Cloud ERP

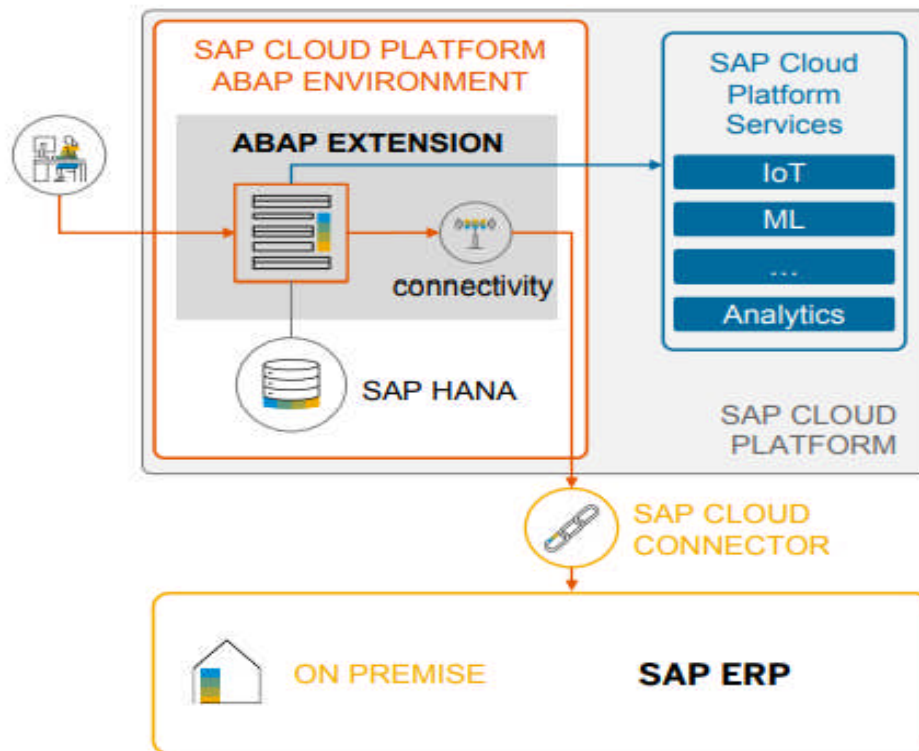


Use SAP Cloud Platform ABAP Environment to extend SAP S/4HANA Cloud or other SAP cloud solutions

- SAP Cloud solutions like SAP S/4HANA Cloud provide in-app extensibility to extend SAP apps and processes, but there is no support for classic custom ABAP development on top of SAP S/4HANA Cloud
- SAP Cloud Platform is the foundation to develop and run custom cloud extensions and the ABAP environment shall be used for ABAP based cloud extensions

SAP Cloud Platform ABAP Environment

- Innovation Platform

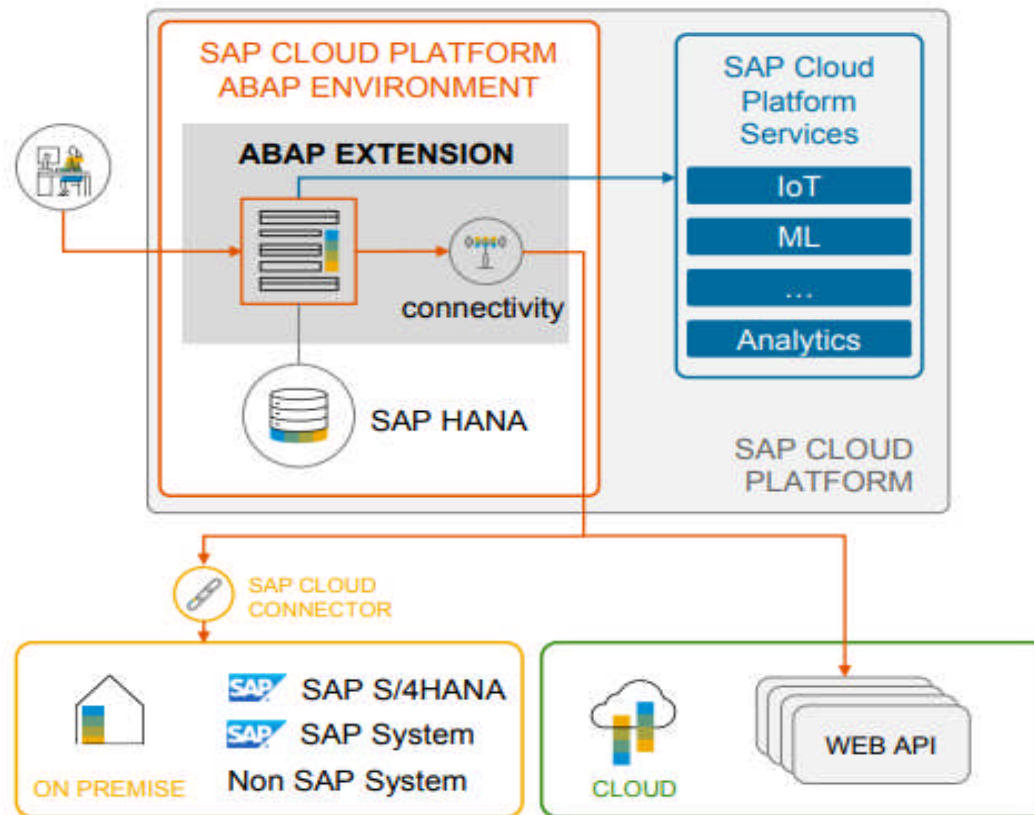


Develop and run innovative ABAP apps on a PaaS in the Cloud

- Benefit from the newest ABAP Platform and SAP HANA database technologies independent from your existing on-premise system landscape
- Build your Fiori apps with the future proof ABAP RESTful Application Programming Model
- Utilize SAP Cloud Platform services like IoT, machine learning etc. in your cloud extension
- Delegate operation of the ABAP PaaS and new technologies to SAP

SAP Cloud Platform ABAP Environment

- Hub-like usage



Decouple ABAP implementations from on-premise core business systems

- EXTERNAL USER GROUP
Make your cloud app available to a broader audience that does not have access to your core business systems (e.g. consumer apps)
- INTEGRATION HUB
Integrate in your cloud extension multiple cloud/on-premise systems with SAP/non-SAP cloud services
- DATA INTEGRATION
Collect data from multiple sources in your cloud extension for further processing and analysis
- DECOUPLED EXTENSION
Cloud extensions use only well defined (remote) APIs of the Business system. This reduces the risk and effort for business system upgrades.



External API Integration

- **HTTPS/OData access to SAP and non-SAP systems**

The ABAP environment supports secure communication with external systems using HTTPS and the OData protocol. This allows RAP applications to consume or expose services to both SAP and third-party systems over the internet.

- **API Hub for ready-to-use interfaces**

SAP API Business Hub provides a central catalog of pre-built APIs from SAP (like S/4HANA, SuccessFactors, etc.) that developers can integrate directly into their ABAP applications to extend functionality without building from scratch.



Debugging and Testing in RAP

- **ADT supports unit tests**

ABAP Development Tools (ADT) in Eclipse allow you to write and run unit tests directly within your development environment, helping you validate small parts of logic automatically.

- **Gateway client for OData testing**

The integrated Gateway Client tool can simulate OData service calls, letting developers test the API behavior of their RAP services without needing a full frontend.

- **Draft preview and transactional tracing**

You can inspect and debug draft data and transactional flows (e.g., create, update) within RAP to ensure correct intermediate states and logic execution.

- **Example for RAP Testing Tools**

TEST-SEAM. " Used to replace real code blocks with testable logic during unit tests

TEST-INJECTION. " Allows injecting mock data or test doubles into the logic under test



RAP: Best Practices Summary

- **Model-first approach with CDS**
Start development by defining the data model using CDS views. This ensures consistency, reusability, and alignment with SAP's declarative design principles.
- **Behavior separation with BDEF**
Keep business logic separate from the data model by using behavior definitions (BDEF). This improves clarity, modularity, and makes your code easier to test and maintain.
- **Stateless, service-oriented architecture**
Design applications to be stateless and expose functionality through standardized services (OData). This enables scalability and aligns with cloud-native principles.
- **Adopt managed BOs when possible**
Use managed business objects to leverage the RAP framework's built-in support for standard operations (CRUD, draft, validations), reducing the need for boilerplate code.

Summary

- RAP enables modern, cloud-ready ABAP development for SAP Fiori and Web APIs
- Follows a layered architecture: CDS (data), BDEF (behavior), OData (service)
- Supports both managed and unmanaged business objects
- Developed using ABAP Development Tools (ADT) in Eclipse
- EML is used for programmatic access to business objects
- Leverages SAP HANA with code pushdown for high performance
- Designed for stateless, scalable, and extensible cloud applications
- UI generation is simplified using annotations and metadata extensions



Source



References

- https://learning.sap.com/courses/building-apps-with-the-abap-restful-application-programming-model/the-big-picture_LE_0e44d65f-a777-4e38-a743-e333c60191a2
- https://learning.sap.com/courses/building-apps-with-the-abap-restful-application-programming-model/the-business-scenario_LE_30426cf5-4115-4fec-a349-b35c8975c279
- https://learning.sap.com/courses/building-apps-with-the-abap-restful-application-programming-model/the-enhanced-business-scenario_LE_1a4a9cd8-d068-4613-95ef-ef05ddf0b3ce
- https://learning.sap.com/courses/building-apps-with-the-abap-restful-application-programming-model/the-business-scenario_LE_4ca2d54b-0ef2-4d81-a83f-ab93c6cdb02d
- https://learning.sap.com/courses/building-apps-with-the-abap-restful-application-programming-model/the-business-scenario_LE_154dc8cd-c3e4-4939-b8b0-8b0b1b326395





Quiz

1. What is the main purpose of the ABAP RESTful Application Programming Model (RAP)?

- a) To replace SAP Fiori entirely
- b) To develop mobile-only applications
- c) To create scalable, cloud-ready applications
- d) To manage SAP system configurations



Answer: C

To create scalable, cloud-ready applications



Quiz

2. What is the role of CDS in RAP?

- a) To manage transport requests
- b) To define business logic rules
- c) To store transactional logs
- d) To define semantic data models



Answer: D

To define semantic data models



Quiz

3. Which component is responsible for exposing data and operations as a service?
- a) Behavior Definition
 - b) CDS View
 - c) Service Definition
 - d) BAPI



Answer: C
Service Definition



Quiz

4. What is the benefit of a managed business object in RAP?

- a) Requires manual implementation of all logic
- b) Supports automatic handling of standard operations
- c) Uses classical Dynpro screens
- d) Disables extensibility



Answer: B

Supports automatic handling of standard operations



Quiz

5. Which tool is used for RAP development in Eclipse?

- a) SE80
- b) SAP GUI
- c) ABAP Development Tools (ADT)
- d) Web IDE



Answer: C

ABAP Development Tools (ADT)

Thank You